

ATK-MS53L0M 激光测距模块使用说明

本应用文档将教大家如何在正点原子开发板上使用 ATK-MS53L0M 激光测距模块。

本文档分为如下几部分：

- 1, ATK-MS53L0M 模块简介
- 2, 硬件连接
- 3, 软件实现
- 4, 验证

1、模块简介

ATK-MS53L0M 激光测距模块是广州市星翼电子科技有限公司（正点原子）新推出的一款 2 米的激光测距模块。串口直接输出测量距离，方便使用。模块可用于无人机、机器人、智能设备等场合。

模块引脚定义如下图 1.1 所示：



名称	功能
红色（VCC）	模块电源，3.3V~5V 输入
黑色（GND）	地线
黄色（TX）	串行数据输出（TX），3.3V TTL 电平
白色（RX）	串行数据输入(RX)，3.3V TTL 电平
蓝色（SCL）	IIC 时钟线（SCL 内部上拉 10K）
绿色（SDA）	IIC 数据线(SDA 内部上拉 10K)

2、硬件连接

说明：本文档以 STM32F103 战舰 V3 开发板为例说明，其他款开发板硬件连接请参考《【图】ATK-MS53L0M 模块与各开发板连接图》。

本实验功能简介：测试模块在 NOLMAL 模式或 MODBUS 模式下测量数据的获取，并将读取的数据和状态输出到串口 1，同时 LED0 在闪烁。

本实验所需要的硬件资源如下

- 1, 战舰 V3 开发板
- 2, 串口 1、串口 3
- 3, ATK-MS53L0M 模块

接下来，我们看看 ATK-MS53L0M 模块与开发板的连接，前面我们介绍了 ATK-MS53L0M 模块的接口，我们通过杜邦线连接模块和开发板的相应端口，连接关系如表 2.1 所示：

ATK-MS53L0M 模块与开发板连接关系						
ATK-MS53L0M 模块	VCC	GND	TXD	RXD		
战舰 V3 开发板	5V	GND	PB11	PB10		

表 2.1 ATK-MS53L0M 模块与战舰 V3 开发板连接关系图
实物连接图如下图 2.2 所示：

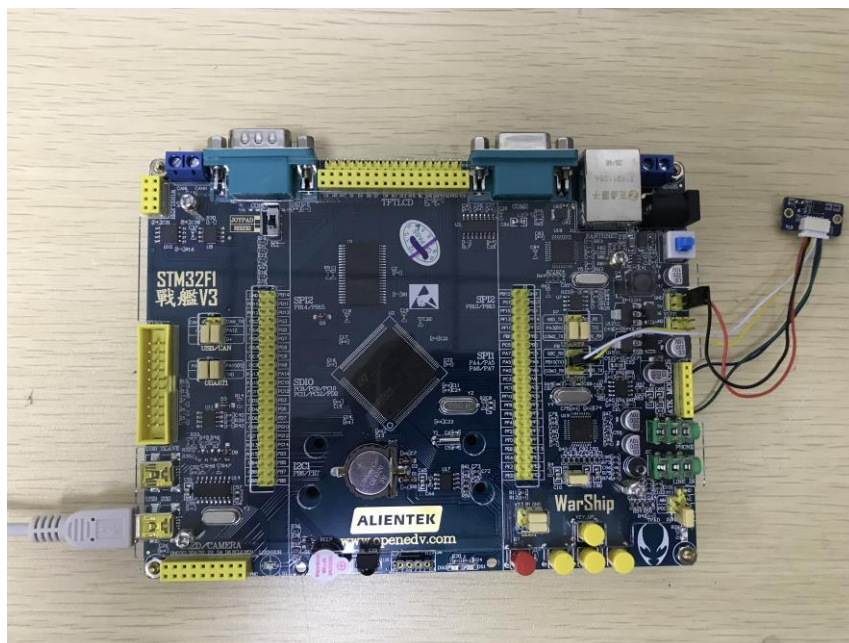


图 2.2 模块与开发板连接

3、软件实现

说明：本文档以 STM32F103 战舰 V3 开发板为例说明，其他开发板软件实现请打开对应的工程源码学习。

本实验例程，我们在 STM32 开发板的程序源码中串口实验库函数板的基础上修改，本章还需要使用串口 3，我们在工程添加了 usart3.c、ms53l0m.c 文件及对应的.h 文件。

usart3.c 主要实现：串口 3 初始化、串口数据中断接收、空闲中断处理、串口数据发送

ms53l0m.c 主要实现：串口数据解包、数据解析、模块功能码读写、模块初始化。

下面我们详细看一下 uart3.c，代码如下：

```
#include "delay.h"
#include "usart3.h"
#include "stdarg.h"
#include "stdio.h"
#include "string.h"
#include "ms53l0m.h"

//串口接收缓存区
u8 USART3_RX_BUF[USART3_MAX_RECV_LEN];
//接收缓冲,最大 USART3_MAX_RECV_LEN 个字节.
u8 USART3_TX_BUF[USART3_MAX_SEND_LEN];
//发送缓冲,最大 USART3_MAX_SEND_LEN 字节
```

```

/*UART3 中断服务函数*/
void USART3_IRQHandler(void)
{
    u8 res;

    if (USART_GetITStatus(USART3, USART_IT_RXNE) != RESET) //接收到数据
    {
        res = USART_ReceiveData(USART3);

        if (!Tof_Data.rx_ok) /*没接收完成*/
        {
            if (Tof_Data.rx_len < (BUFF_MAX_LEN - 1))
            {
                Tof_Data.rx_buff[Tof_Data.rx_len] = res; /*保存数据*/
                Tof_Data.rx_len++; /*接收长度累加*/
            }
            else
            {
                Tof_Data.rx_len = 0; /*接收超过范围*/
                Tof_Data.rx_ok = 1;
            }
        }
    }

    if (USART_GetITStatus(USART3, USART_IT_IDLE) != RESET) //空闲中断
    {
        res = USART_ReceiveData(USART3); /*清空空闲中断*/
        if ((Tof_Data.rx_len != 0) && !Tof_Data.rx_ok)
        {
            Tof_Data.rx_ok = 1; /*标志接收完*/
        }
    }
}

USART_InitTypeDef USART_InitStructure;
//初始化 IO 串口 3
//pclk1:PCLK1 时钟频率(Mhz)
//bound:波特率
void usart3_init(u32 bound)
{
    NVIC_InitTypeDef NVIC_InitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE); // GPIOB 时钟

```

```

RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART3, ENABLE); //串口 3 时钟使能

USART_DeInit(USART3); //复位串口 3
//USART3_TX    PB10
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10; //PB10
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
GPIO_Init(GPIOB, &GPIO_InitStructure); //初始化 PB10

//USART3_RX    PB11
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; //上拉输入
GPIO_Init(GPIOB, &GPIO_InitStructure); //初始化 PB11

USART_InitStructure.USART_BaudRate = bound; //波特率一般设置为 9600;
USART_InitStructure.USART_WordLength = USART_WordLength_8b;
//字长为 8 位数据格式
USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
USART_InitStructure.USART_HardwareFlowControl
    = USART_HardwareFlowControl_None; //无硬件数据流控制
USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式

USART_Init(USART3, &USART_InitStructure); //初始化串口 3

USART_Cmd(USART3, ENABLE); //使能串口

//使能接收中断
USART_ITConfig(USART3, USART_IT_RXNE, ENABLE); //开启接收中断
USART_ITConfig(USART3, USART_IT_IDLE, ENABLE); //开启空闲中断

//设置中断优先级
NVIC_InitStructure.NVIC_IRQChannel = USART3_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2; //抢占优先级 3
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3; //子优先级 3
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ 通道使能
NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器

Tof_Data.rx_ok = 0; //清零

}

//串口 3,printf 函数
//确保一次发送数据不超过 USART3_MAX_SEND_LEN 字节

```

```

void u3_printf(char *fmt, ...)
{
    u16 i, j;
    va_list ap;
    va_start(ap, fmt);
    vsprintf((char *)USART3_TX_BUF, fmt, ap);
    va_end(ap);
    i = strlen((const char *)USART3_TX_BUF); //此次发送数据的长度

    for (j = 0; j < i; j++) //循环发送数据
    {
        while (USART_GetFlagStatus(USART3, USART_FLAG_TC) == RESET);
        //循环发送,直到发送完毕

        USART_SendData(USART3, USART3_TX_BUF[j]);
    }
}

//串口 3 数据发送
//*date:数据缓存地址
//len:数据长度
void usart3_send(uint8_t *data, uint8_t len)
{
    uint8_t i = 0;

    for (i = 0; i < len; i++) //循环发送数据
    {
        while ((USART3->SR & 0X40) == 0); //循环发送,直到发送完毕
        USART3->DR = *(data + i);
    }
}

```

这部分代码，主要实现了串口 3 的初始化，串口 3 发送数据，中断接收数据，以及空闲中断处理的功能。串口 3 串口接收到的数据存储到 `Tof_Data.rx_buff` 数组，串口没接收到数据空闲，触发空闲中断，将 `Tof_Data.rx_ok` 置位，表示以接收完数据。

下面我们看一下 `ms53l0m.c` 和 `ms53l0m.h`，由于代码过长，这里就不详细贴出，代码折叠如下图 3.1 和 3.2 所示：

```

26 #include "usart3.h"
27 #include "string.h"
28 #include "usart.h"
29 #include "delay.h"
30 #include "led.h"
31
32 _Tof_Info Tof_Info;
33 _Tof_Data Tof_Data;
34
35 /**
36  * @brief 计算CRC校验和
37  * @param *buff:缓存首地址
38  * @param len:数据长度
39  * @retval uint16_t: CRC校验和值
40  */
41 inline static uint16_t CRC_Check_Sum(uint8_t* buff, uint16_t len)
42 {
43
44 }
45 /**
46  * @brief 帧数据包解析
47  * @param data:读取的数据
48  * @retval uint8_t: 工作状态:0, 成功>0, 失败
49  */
50 inline static uint8_t Ms5310m_Unpack(uint16_t* data)
51 {
52
53 }
54 /**
55  * @brief 从模块功能码读数据
56  * @param addr:设备地址
57  * @param reg:功能码地址
58  * @param datalen:数据长度:只能是1或2
59  * @param data:读取数据
60  * @retval uint8_t:
61  */
62 uint8_t Ms5310m_RData(uint16_t addr, uint8_t reg, uint8_t datalen, uint16_t* data)
63 {
64
65 }
66 /**
67  * @brief 向模块功能码写数据
68  * @param addr:设备地址
69  * @param reg:功能码地址
70  * @param data:写入数据
71  * @retval uint8_t: 应答包返回状态
72  */
73 uint8_t Ms5310m_WData(uint16_t addr, uint8_t reg, uint8_t data)
74 {
75
76 }
77 /**
78  * @brief 传感器初始化
79  * @param
80  * @retval
81  */
82 void Ms5310m_Init(void)
83 {
84
85 }
86 /**
87  * @brief Modbus模式数据获取
88  * @param
89  * @retval
90  */
91 void Modbus_DataGet(void)
92 {
93
94 }
95 /**
96  * @brief Normal模式数据获取
97  * @param
98  * @retval
99  */
100 void Normal_DataGet(void)
101 {
102
103 }

```

图 3.1 ms5310m.c 源码图

```

31 #define Uart_Init(bps).....usart3_init(bps)...../*串口初始化API*/
32 #define Uart_Send(data, len).....usart3_send(data, len)...../*串口发送API*/
33
34 #define MFRAME_H.....0X51/*主机请求帧头*/
35 #define SFRAME_H.....0X55/*从机应答帧头*/
36 #define SENSOR_TYPE.....0X0A/*MS53L0M传感器*/
37
38
39 #define RPACK_MAX_LEN.....270/*接收包最大长度*/
40 #define RPACK_MIN_LEN.....8/*接收包最小长度*/
41
42 #define BUFF_MAX_LEN.....300/*串口接收最大长度*/
43
44 /*用户数据处理*/
45 typedef struct
46 {
47     uint8_t tx_buff[BUFF_MAX_LEN];/*发送缓存*/
48     uint8_t rx_buff[BUFF_MAX_LEN];/*接收缓存*/
49     bool rx_ok;...../*接收完成标志:1:完成:0:还没*/
50     uint16_t rx_len;...../*接收缓存的长度*/
51 } _Tof_Data;
52
53
54 extern _Tof_Data Tof_Data;
55
56 /*模块参数表*/
57 typedef struct
58 {
59     /*模块功能码*/
60     enum
61     {
62         /*系统设置参数*/
63         enum
64         {
65             /*回传速度参数*/
66             enum
67             {
68                 /*串口波特率参数*/
69                 enum
70                 {
71                     /*测量模式参数*/
72                     enum
73                     {
74                         /*工作模式参数*/
75                         enum
76                         {
77                             /*错误帧输出设置*/
78                             enum
79                             {
80                                 /*应答包返回状态*/
81                                 enum
82                                 {
83                                     /*
84                                     */
85                                     void Ms53l0m_Init(void);
86                                     uint8_t Ms53l0m_WData(uint16_t addr, uint8_t reg, uint8_t data);
87                                     uint8_t Ms53l0m_RData(uint16_t addr, uint8_t reg, uint8_t datalen, uint16_t *data);
88                                     void Modbus_DataGet(void);
89                                     void Normal_DataGet(void);
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157

```

图 3.2 ms53l0m.h 源码图

如上图所示，源码中 `uint8_t Ms53l0m_Unpack(uint16_t *data)` 函数将串口 3 接收的数据按照协议进行解析，解析成功，若是读操作数据会保存在 `data`，读写操作函数都会返回数据包解析状态。

源码中 `Ms53l0m_Wdata` 和 `Ms53l0m_Rdata` 函数用于读写模块功能码，功能码地址在 `ms53l0m.h` 中。`Ms53l0m_Init` 函数主要是初始化通信串口波特率（**注意：请确保模块的串口波特率，以免开发板与模块通信异常**），`Modbus_DataGet` 函数让模块工作在 `Modbus` 模式下，通过读功能码形式，获取距离，通过串口 1 输出。而 `Normal_DataGet` 函数让模块工作在 `Normal` 模式下，解析回传的数据，获取距离，并且通过串口 1 进行输出。关于模块功能码的详细说明请参考《[ATK-MS53L0M 激光测距模块用户手册.pdf](#)》。

下面我们看一下 `main.c`，代码如下：

```

int main(void)
{
    delay_init();                //延时函数初始化

```

```

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
//设置中断优先级分组为组 2：2 位抢占优先级，2 位响应优先级
uart_init(115200);           //串口初始化为 115200
LED_Init();                 //LED 初始化
Ms53l0m_Init();             //MS53L0M 模块初始化
while(1)
{
    //Normal_DataGet(); /*NORMAL 模式获取距离数据*/
    Modbus_DataGet(); /*MODBUS 模式获取距离数据*/
}
}

```

此部分代码好简单，外设和模块的初始化，然后在主循环选择对应的模式进行测试。接下来看看代码验证。

4、验证

特别注意：请确保开发板的 USART1 的两个跳线帽短接（TXD 与 PA10 短接，RXD 与 PA9 短接），USB 数据线已经插入 USB_SLAVE 口，否则调试信息无法输出到串口调试助手。

在代码编译成功之后，我们下载代码到我们的 STM32 开发板上（ATK-MS53L0M 模块已经连接上开发板），打开串口调试助手（设置波特率 115200），Modbus 模式测试显示如下图 4.1 所示：

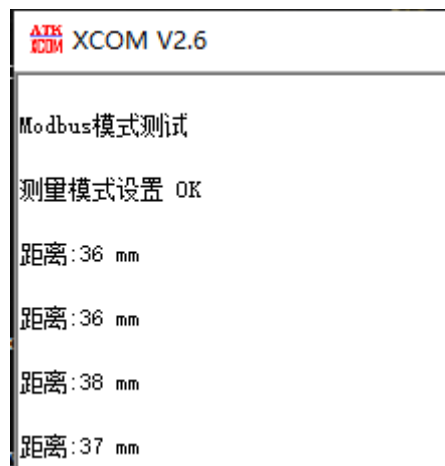


图 4.1 Modbus 模式测试效果

Normal 模式测试，如下图 4.2 所示。

ALIENTEK XCOM V2.6

Normal 模式测试

回传速率设置 OK

测量模式设置 OK

工作模式设置 OK

距离: 36mm

距离: 37mm

距离: 36mm

距离: 35mm

图 4.2 Normal 模式测试

正点原子@ALIENTEK

2021-3-26

公司网址: www.alientek.com

技术论坛: www.openedv.com

电话: 020-38271790

传真: 020-36773971

